Computational Linguistics - 2

CL is the application of CSc in Linguistics. Includes a lot of theoretical questions - NLP has a more applied focus

Three eras of CL

1) Rule based approaches
2) Statistical methods
3) Neural models (Deep Learning)

Ambiguity:- Can be at word, phrase or sentence levels

Words are challenging

→ Segmenting into words

→ Domain specific meanings etc

Origin of He/Him : She/Her → Bantu langs

Manning and Schulze

Zipf's Law:-

Tokens:- Individual occurences of words

Types:- No. of unique instances

a rose is a rose is a rose

Types = a, rose, is

Tokens = 8

Type-token ratio is the measure of lexical diversity.

Hapax Legomena :- Words that appear only once.

Common 100 words usually account for > 50% of tokens.

Zipf's Law = $\underset{\text{freq}}{\nu} \propto \frac{1}{\text{rank}}$

Speaker prefers a smaller vocab of common words for easier comm.
Hearer prefers a larger vocab of rarer words for lucid comm.

Thus the two arrive at a maximally economical compromise

ZL -> $f \propto \frac{1}{r}$ => $f \cdot r = k$ (const)

Mandelbrot in 1956 argued that this was bad fit for both the low and the high ranks. He instead suggested

$f = P(r+\rho)^{-\beta}$ where P, ρ and β are text based params.

If $\beta = 1$ & $\rho = 0$, Mandelbrot's law becomes Zipf's law.

There is a lot of variation b/w the different types of text, hence parsers trained on one type would usually not work for other types

Supposing we randomly generate text, it will exhibit Zipf's Law.

The probability of word length n = $\underbrace{\left(\frac{26}{27}\right)^n}_{\text{non blank character}} \times \underbrace{\frac{1}{27}}_{\text{followed by a blank}}$

Text Classification:-

Document Classification:- Sort documents into user defined classes

Sentiment Analysis:- Assigning ~~better~~ sentiment to a text. Initially a ternary system:- +ve, -ve & neutral, but now has 9 types.

Authorship Attribution:- Author identification / Plagiarism

Spam Filtering:- Spam vs Ham

Language Identification:- closed-world domain

Assumption :- Single source, monolingual documents of certain length where we know every language

n-grams:- continuous n-sized sequences of words or characters

Store a frequency ~~of~~ distribution of trigrams for every given language. Apply the freq dist to a new text and use it to judge the source language

But cross domain performance is much (much!) poorer than ~~intra~~- in-domain performance

Our goal is to identify n-grams with high and low language association and low domain association

Trigrams ~~*~~ are the tradeoff b/w paucity and reliability

Higher n-grams are rare but reliable and vice-versa

Information Gain → or called Entropy (degree of uncertainity)

$$\text{Info}(D) = -\sum_{i=1}^{m} p_i \log_2 (p_i)$$

m → Total no. of classes

p_i → Non zero probability that any tuple in D belongs to class C_i

Info(D) → Avg information to identify the class label of a tuple d

Dataset → Tuple with class label

Training Samples

Attribute or feature A having v distinct values $\{a_1, \dots, a_n\}$

Unit of entropy is bits

To encode n different sequences is $\lceil \log_2 n \rceil$

But info. is related to probability. $p \propto \frac{1}{\text{info}}$

Entropy in a sense a measure of impurity of data (mixing of classes, imbalance in classes)

High Entropy is better for training data

Supervised Learning :- Training & test data have been labelled with the correct answers

$$\text{Info}_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times \text{Info}(D_j)$$

→ no of ~~types~~ times | cardinality?

↓

Attribute $A = \{a_i\} \forall i \in [1, v]$

↳ values

$$\text{Gain}(A) = \text{Info}(D) - \text{Info}_A(D)$$

Information Gain tells us how important a given attribute of the feature vector is.

In multiclass cases, the entropy would increase.

multiclass cases can be thought of as a recursive two-class cases

In multilingual documents, language identification is much harder (esp in code switched & code mixed data)

Spam Identification:-

spam = emails that we don't want, incl unsolicited mails

Spam filters help classify spam/ham.

Rule based filters :- Useful only if patterns can be identified, and keep changing along with the spammers

Statistical filter:- Count freq of words appearing in ham mails & spam mails and calculate the spam probability of a word. We provide the model with a large set of examples.

$$\mathbb{P}(\text{spam} \mid \text{Document}) = \frac{\mathbb{P}(D \mid \text{spam}) \times \mathbb{P}(\text{spam})}{\mathbb{P}(D)}$$

Probability :-

Experiment:- Process by which observations are made (Trials)

Sample Space Ω = Collection of events / basic outcomes

Countably ∞ → Discrete

Uncountably ∞ → Continuous

A set S is countable if $\exists$ an inj mapping from S to $\mathbb{N}$

Discrete Probability fn :- distributes a probability mass of 1 throughout Ω ; $P: F \to [0,1]$

Event Space :- $\mathcal{P}(\Omega)$ if Ω is discrete, $\mathcal{B}(\Omega)$ if Ω is continuous

Random Variable :- $X: \Omega \to \mathbb{R}^n$ where n is usually 1

Discrete Random Variables : $X: \Omega \to S$ where S is a countable subset of $\mathbb{R}$

If $X: \Omega \to \{0,1\}$ then X is the indicator random variable or Bernoulli Trials

Probability Mass fn

$$\mathbb{P}(X=k) = p_X(x)$$

Probability Space

P on sample space Ω, fn from $E \subseteq \Omega$ to $\mathbb{R}$ s.t.

1) Non-negativity : $\mathbb{P}(E) \geq 0 \ \forall E \subseteq \Omega$

2) Disjoin union :- If E_1, E_2 are disjoint, then $\mathbb{P}(E_1 \cup E_2) = \mathbb{P}(E_1) + \mathbb{P}(E_2)$

3) Properness :- $\mathbb{P}(\Omega) = 1$

Conditional Probability :-

$$\frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} = \mathbb{P}(A|B)$$

Chain Rule

$$\mathbb{P}(A_1 \cap \dots \cap A_n) = \mathbb{P}(A_1) \times \mathbb{P}(A_2|A_1) \times \dots \mathbb{P}(A_n | \bigcap_{i=1}^{n-1} A_i)$$

Independent Events

$$\mathbb{P}(A \cap B) = \mathbb{P}(A) \times \mathbb{P}(B)$$

$$\Leftarrow \mathbb{P}(A \cap B | C) = \mathbb{P}(A|C) \times \mathbb{P}(B|C)$$

$$\underbrace{\mathbb{P}(A|B)}_{\text{Posterior}} = \frac{\overbrace{\mathbb{P}(B|A)}^{\text{Likelihood}} \times \overbrace{\mathbb{P}(A)}^{\text{Prior}}}{\underbrace{\mathbb{P}(B)}_{\text{Normalizing Const}}}$$

Sentiment Analysis:- Analysis of the attitude of a document.

~~Input:- a document~~ d

Rules based on combinations of words & features

Input:- a doc d

fixed set of classes $C = \{c_1, \dots, c_j\}$

a training set of m hand labelled docs $((d_1, c_1), (d_2, c_2), \dots (d_m, c_m))$

Output:- A learned classifier $y: d \to c$

Classifier

→ Generative ⇒ Naive Bayes, HMM ⇒ Model of how a class could generate input data. Given an observation, which class generated that observation

...

→ Discriminative ⇒ Model of features which discriminate b/w classes
⇒ Logistic Regression

Naive Bayes Intuition:-

① Uses a bag of words ⇒ Doc where only (word, freq) is stored.

For a doc d & a class c

$$\mathbb{P}(c|d) = \frac{\mathbb{P}(d|c) \times \mathbb{P}(c)}{\mathbb{P}(d)}$$

Our estimate of the correct class

Max a posteriori class

$$\hat{c} = \operatorname{argmax}(\mathbb{P}(c|d)) \text{ where } c \in C \text{ \& } C = \{c_1, \dots c_n\}$$

$$= \operatorname{argmax}\left(\frac{\mathbb{P}(d|c) \times \mathbb{P}(c)}{\underbrace{\mathbb{P}(d)}_{\text{common for both } c}}\right) \forall c \in C$$

$$\cong \underset{c \in C}{\operatorname{argmax}} \; P(d|c) \times P(c)$$

Every doc is represented as a set of features $\{f_1, \dots f_n\}$

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \; \underbrace{P(f_1 \dots f_n | c)}_{\text{Likelihood}} \times \overbrace{P(c)}^{\text{Prior}}$$

② Conditional Independence assumption

$$P(f_1 \dots f_n | c) = P(f_1|c) \times P(f_2|c) \times \dots \times P(f_n|c)$$

$$\hat{c} = C_{NB} = \underset{c \in C}{\operatorname{argmax}} \; \prod_{f \in F} (P(f|c)) \times P(c)$$

$P(c) = \frac{N_c}{N_{doc}}$ → No of docs with class c

Maximum Likelihood Estimate (MLE) → Concatenate all documents of class c into one document. Words as features

$$P(w_i|c) = \frac{\text{count}(w_i, c)}{\sum_{w \in V} \text{count}(w, c)}$$

V → Union of word types in all classes

Smoothing:- $P(w_i|c) = \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} (\text{count}(w_0, c) + 1)}$

↓

Laplace

↳ Add one smoothing

Markov Assumption:- Probability of a word depends only on the previous word.

↳ Will face underflow, hence we take log.

Instead of smoothing, we remove words with 0 probability

Stopword list:- Top 10-50 words in the vocab sorted by freq.

↳ Very frequent words like a & the.

Lexicon:- Pre-built word list

Naive Bayes :: better than decision trees because dts are prone to fragmentation

Sir shared slides

Reference Resolution

Coreference:- Two mentions refers to the same entity

Anaphora:- When a Term refers to another Term (antecedent)

Friday's class Class by?

Not all Anaphora is coreferential.

Cataphora:- mentioned before the antecedent/referents

Mentions:- Span of text referencing some entity :- pronouns, NE, NP, ...

named entity

But marking all as mentions causes over-generation.

For ex:- It has been raining

Four kinds of coreference models:-

Rule Based, ML models (Mention Pair, Mention ranking, Clustering based)